

Система автоматизированного обогащения товарных данных для интернет-магазина

Фролов М. А.

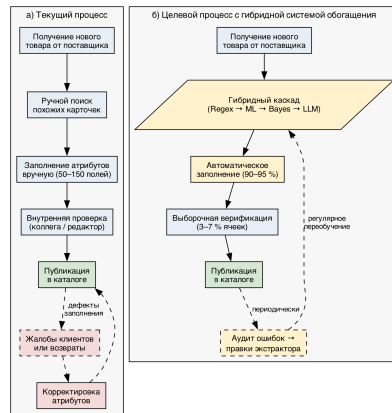
Московский авиационный институт
Институт № 8, кафедра 806 «Вычислительная математика и программирование»
Научный руководитель: Судаков В. А.

Москва, 2026

Проблемные вопросы практики обогащения карточек

- ▶ Каталоги маркетплейсов — **десятки миллионов SKU**.
- ▶ Партнёры присылают **лишь базовый набор** полей.
- ▶ PIM-системы **хранят, но не извлекают** атрибуты.
- ▶ Ручное заполнение — **узкое место** вывода ассортимента.

$\sim 10^7$ SKU · часы на карточку ·
линейная стоимость от объёма



Актуальность направления и задача ВКР

Противоречия

- ▶ **Процесс** — 10^7 + SKU vs пропускная способность контент-менеджеров.
- ▶ **Средства** — PIM хранит, но не извлекает атрибуты.
- ▶ **Технология** — точность БЯМ vs её **цена** на промышленных объёмах.

Задача ВКР. Программное средство обогащения товарных данных с заданной точностью при **кратном сокращении стоимости** относительно прямого вызова большой языковой модели и допускающее локальное развёртывание.

*⇒ перенос акцента с «одна большая модель на всё» на
поатрибутную маршрутизацию между слоями разной стоимости.*

Аналоги: пробел в существующих подходах

Критерий	TXtract	MAVE	FrugalGPT	AutoMix	Наша система
Per-attribute policy	✓	✓	—	—	✓
Калиброванный отказ от ответа	—	—	—	✓	✓
Cost-aware routing	—	—	✓	✓	✓
Непересекающиеся бренды	—	—	—	—	✓
Гибридная архитектура	—	—	—	—	✓
Локальное развёртывание	✓	✓	—	—	✓

Ни одна из существующих систем не объединяет **per-attr policy**, **проверку на непересекающихся брендах** и **гибридную архитектуру** одновременно.

Цель и задачи ВКР

Цель. Программное средство обогащения товарных данных на основе гибридной каскадной архитектуры.

Частные задачи

1. Анализ существующих подходов, выявление пробела.
2. Функциональные, нефункциональные, архитектурные требования; интегральный критерий качества.
3. Каскад из 4 слоёв + поатрибутная политика маршрутизации.
4. Реализация end-to-end демо-комплекса (REST API + Web-UI).
5. Pre-registered протокол оценки на выборке с непересекающимися брендами.
6. Рекомендации по внедрению и эксплуатации.

Объект, предмет и рабочее предположение

Объект. Процессы обогащения карточек товаров в интернет-магазинах.

Предмет. Методы и алгоритмы извлечения атрибутов, политики маршрутизации между слоями разной стоимости и протоколы их оценки.

Рабочее предположение.

- ▶ **Каскад** из слоёв разной стоимости (regex → ML → Bayes → LLM) обеспечивает заданную точность обогащения товарных данных.
- ▶ **Поатрибутная политика эскалации** по калиброванной уверенностикратно сокращает стоимость относительно прямого вызова большой языковой модели.
- ▶ **Локальное развёртывание** возможно на коммерчески доступной инфраструктуре.

Исходные посылки

1. Унаследованная инфраструктура

PIM (Akeneo, Pimcore, Salsify) хранит, но не извлекает атрибуты.

2. Теоретическая основа

Многоязычные эмбединги, калиброванная классификация (Platt + isotonic), байесовские сети, prompt-инженерия (few-shot, schema-constrained).

3. Технологический стек

Python / FastAPI / Go-шлюз / React; Docker-compose; XGBoost, sentence-transformers, pgmpy; Ollama для локальной языковой модели.

4. Открытые данные

Open Food Facts (4,5 млн SKU, ~40 % FR / 10 % EN / 7 % DE) + открытые источники по электронике для холодного старта.

Методы исследования

Системный анализ

Декомпозиция процесса, выявление противоречий.

Управление требованиями

ФР / НФР / А-требования, интегральный критерий Q.

Машинное обучение

Эмбединги + XGBoost, калибровка Platt + isotonic.

Промпт-инженерия

Few-shot со схемой ответа, селективная эскалация.

Предрегистрируемый эксперимент

Непересекающиеся бренды, McNemar + Бонферрони, Wilson 95 % CI.

Итеративная разработка

Цикл

аудит → правки → переобучение → измерение.

Основные результаты работы

1. **Требования и качество:** 6 ФР, 4 НФР, 3 А; критерий Q с 4 показателями.
2. **Методика:** 4-слойный каскад с поатрибутной политикой эскалации.
3. **Архитектура:** REST-микросервис, упакованный в `docker-compose`.
4. **Реализация:** ~15 тыс. строк Python, 22 пары «категория-атрибут», работающий e2e демо.
5. **Оценка:** достигнуты целевые показатели качества (точность, сокращение стоимости, покрытие, калибровка) — детали далее.

Формальная постановка задачи

Вход:

$x = (\text{product_name}, \text{brands}, \text{ingredients_text}, \text{quantity}, \text{code})$.

Политика обогащения:

$$\pi : x \longrightarrow (a, \hat{y}, c, \ell)$$

a — атрибут, $\hat{y}$ — значение, $c \in [0, 1]$ — **калиброванная** уверенность, ℓ — слой-источник.

Метрики:

- ▶ $\text{Ass}(\pi)$ на проверке с непересекающимися брендами.
- ▶ $\text{C}(\pi)$ — стоимость относительно прямого вызова большой языковой модели.
- ▶ Покрытие каскадом без обращения к языковой модели.

ℓ и c позволяют верифицировать выборочно.

Состав требуемых функций программного средства

Функциональные требования

- ▶ **F1.** Предсказание значения каждого целевого атрибута.
- ▶ **F2.** Возврат *слоя-источника* ($L_1 / L_2 / L_3 / L_4$).
- ▶ **F3.** Возврат *калиброванной уверенности* $c \in [0, 1]$.
- ▶ **F4.** Селективная эскалация на запасной слой языковой модели.
- ▶ **F5.** Байесовская валидация: флаг подозрительных решений.
- ▶ **F6.** Поатрибутная настройка через конфигурацию.

Нефункциональные требования

- ▶ **N1.** Доля вызовов языковой модели $\leq 10\%$ решений.
- ▶ **N2.** Локальное развёртывание (без внешних API).
- ▶ **N3.** Обобщение на непересекающихся брендах.
- ▶ **N4.** Воспроизводимость: предрегистрация + `reproduce.sh`.

Архитектурные требования. **A1.** Контейнеризация (docker-compose). **A2.** Стабильный REST-контракт между слоями. **A3.** Холодный старт: новая категория без правки ядра.

Система качества продукта и интегральный критерий

Качество описано **четырьмя показателями** (точность, стоимость, покрытие, калибровка), сведёнными в интегральный критерий Q:

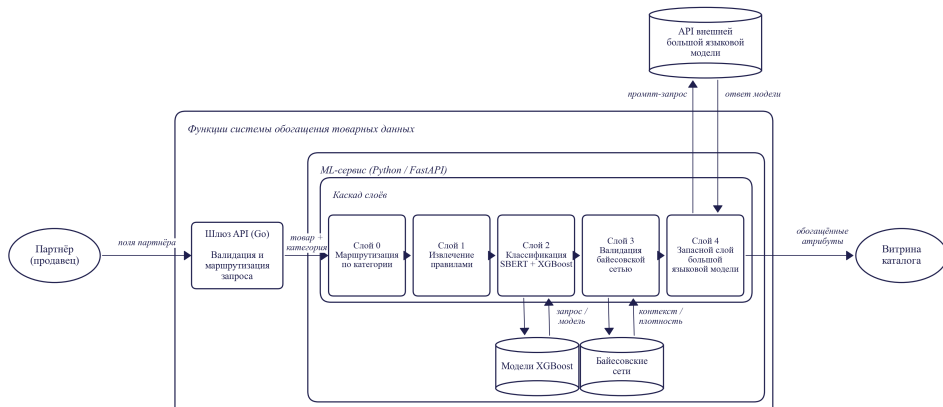
$$Q = w_1 \cdot \text{Acc} + w_2 \cdot \left(1 - \frac{C_{\text{LLM}}}{C_{\text{max}}}\right) + w_3 \cdot \text{Cov} - w_4 \cdot \text{ECE}$$

Acc — точность на непересекающихся брендах; $C_{\text{LLM}}/C_{\text{max}}$ — доля LLM-вызовов; Cov — покрытие каскадом до LLM; ECE — ошибка калибровки.

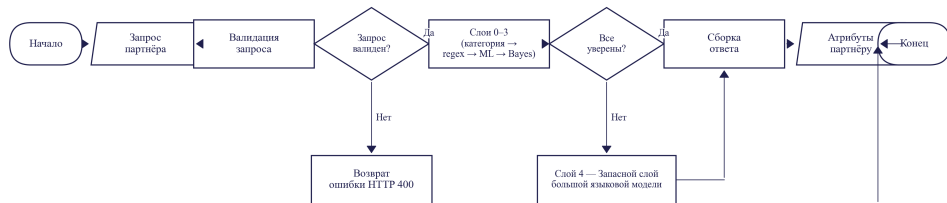
Веса: $w_1 = 0,40$, $w_2 = 0,30$, $w_3 = 0,20$, $w_4 = 0,10$ (приоритет точности над стоимостью согласован с типовыми требованиями РИМ-индустрии).

Показатель	Целевое	Достигнуто
Точность каскада	$\geq 90\%$	94,8 %
$C_{\text{LLM}}/C_{\text{max}}$	$\leq 0,10$	0,071
Cov (покрытие до LLM)	$\geq 0,90$	0,929
ECE (калибровка)	$\leq 0,05$	per-attr ниже целевого

Функциональная модель системы



Ключевые алгоритмы каскада



Решение слоя: $\hat{y}$ принимается, если $p_{\max}(x) \geq \theta_{c,a}$ (поатрибутный порог, калибровка Platt + isotonic). Иначе — эскапация. **Языковая модель** вызывается на 7.1 % ячеек

Данные и протокол оценки

Источник: Open Food Facts, 4,5 млн SKU (~40 % FR / 10 % EN / 7 % DE).

3 категории: паста (8 атр.), шоколад (7), сыры (7) — **22 пары** «категория-атрибут».

Эталон: два уровня контрольной разметки — основной ($n = 3257$) и проверочный ($n = 615$, нижняя оценка точности).

Протокол: проверка на
непересекающихся брендах.

Предрегистрация гипотезы о маршрутизаторе: cd9ac7a (2026-05-13).

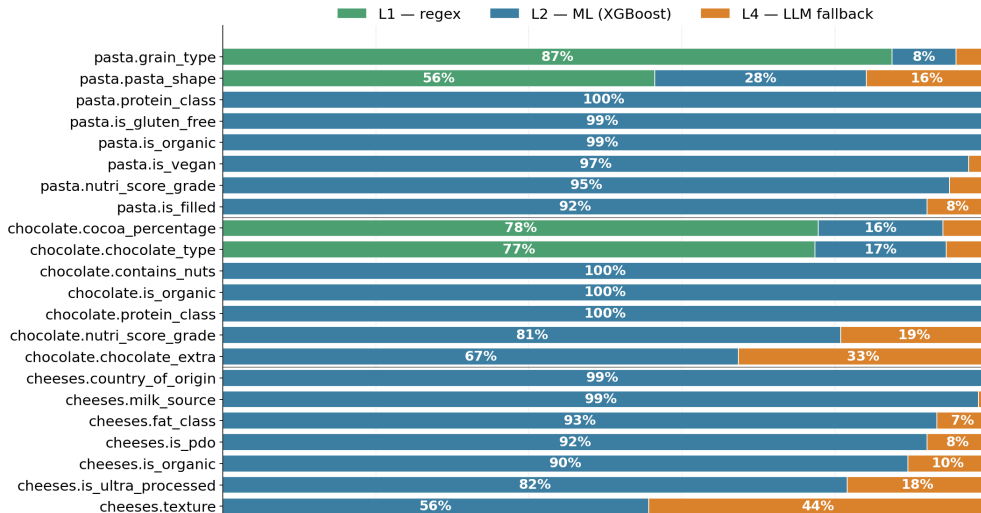
Категория	Атрибутов
Pasta	8
Chocolate	7
Cheeses	7
<i>Расширения:</i>	
Beverages	8
Cereals	8
Cosmetics	6
<i>Холодный старт:</i>	
Electronics	8

Главный результат: точность и стоимость

Метрика	Значение	<i>n</i>
Точность каскада (микро)	94,8 %	3257
Макро-F1 каскада	0,899	3257
Точность маршрутизатора категории	95,4 %	570
Покрытие сквозной цепочки	96,0 %	3257
Точность производственной цепочки	91,1 %	3257
Нижняя оценка (проверочная выборка)	87,5 %	615

Проверка на непересекающихся брендах. **Доля языковой модели: 7,1 %** ячеек ⇒ сокращение стоимости в **14×** раз.

За счёт чего: вклад слоёв по 22 атрибутам



Результаты разработки программного средства

1. **Соответствие требованиям.** Все 6 ФР, 4 НФР, 3 А реализованы; **4/4** показателя качества достигли целевых.
2. **Объём.** ~15 тыс. строк Python; 35+ модулей; 7 категориальных схем; 22 пары.
3. **Демо.** `docker-compose`: React + Go + FastAPI; 5 REST-эндпоинтов; ~200 мс на запрос.
4. **Воспроизводимость.** ~70 тестов; ноутбук как исполняемый §3.3; `reproduce.sh` восстанавливает headline-цифры.

Научная и практическая значимость

Научная значимость.

- ▶ Методика построения гибридного каскада с **поатрибутной калиброванной уверенностью** и **предрегистрируемым** протоколом оценки.
- ▶ Предрегистрированный **отказ** от гипотезы об обучаемом маршрутизаторе в пользу статической политики (простота и предсказуемость).
- ▶ Воспроизводимость методики подтверждена на новой категории (электроника) в режиме холодного старта.

Практическая значимость.

- ▶ Работающее ПО в `docker-compose`, интегрируется рядом с PIM.
- ▶ Кратное сокращение стоимости и ручной нагрузки контент-менеджеров.
- ▶ Возможность локального развёртывания — независимость от внешних API.

Достоверность и обоснованность результатов

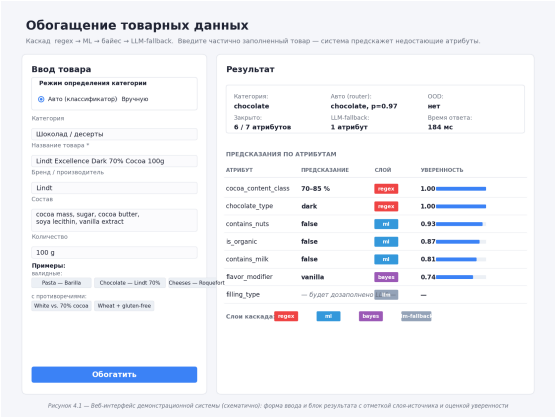
Достоверность:

- ▶ проверка на непересекающихся брендах: основная выборка $n=3257$ + проверочная $n=615$ (нижняя оценка);
- ▶ **предрегистрация** протокола гипотезы о маршрутизаторе в git cd9ac7a до создания артефакта;
- ▶ Wilson 95 % CI на всех headline-числах;
- ▶ McNemar + поправка Бонферрони ($\alpha/3$);
- ▶ воспроизведение через `reproduce.sh`.

Обоснованность. Признанные методы: sentence-transformers, XGBoost, байесовские сети, калибровка Platt + isotonic; сравнение с TXtract, MAVE, OpenTag, FrugalGPT, AutoMix.

Реализация. `docker-compose` stack с REST API и Web-UI; код и `reproduce.sh`

Демонстрационный комплекс — экранные формы



Рекомендации по внедрению, эксплуатации и развитию

Внедрение

- ▶ `docker-compose` up рядом с PIM.
- ▶ Стартовый серебряный эталон из истории каталога.
- ▶ Поатрибутные пороги через JSON-конфиги.
- ▶ Языковая модель: внешний API или локальный Ollama.

Эксплуатация

- ▶ Мониторинг доли языковой модели ($\leq 10\%$).
- ▶ Точность на выборке $n \geq 200$ еженедельно.
- ▶ Аудит при росте байесовских флагов $> 5\%$.
- ▶ Верификация только отмеченных ячеек.

Развитие

- ▶ Холодный старт на новые категории.
- ▶ Расширение схем атрибутов.
- ▶ Замена базовой модели эмбедингов на более крупную.
- ▶ Подключение более дешёвых языковых моделей.

Оперативный эффект от внедрения

94,8 %

точность каскада
($n=3257$)

91,1 %

точность
производственной
цепочки

7,1 %

доля ячеек,
уходящих в языковую
модель

14×

сокращение стоимости
относительно прямого
вызова

⇒ кратное сокращение ручной нагрузки и стоимости обогащения
при сохранении локального развёртывания.

Заключение: задачи и полученные результаты

Задача	Результат
1. Анализ подходов	Пробел: гибридная архитектура + поатрибутная политика + предрегистрируемая оценка.
2. Требования и качество	6 ФР, 4 НФР, 3 А; интегральный критерий Q — все цели достигнуты.
3. Каскад и маршрутизация	4 слоя, 22 пары × 3 категории, калиброванные пороги.
4. Реализация	<code>docker-compose</code> + REST + веб-интерфейс; ~15 тыс. строк Python.
5. Предрегистрируемая оценка	$n=3257$: точность каскада 94,8 % / 0,899 макро-F1, производственная цепочка 91,1 % , 14× дешевле.
6. Рекомендации	Внедрение / эксплуатация / развитие; холодный старт.

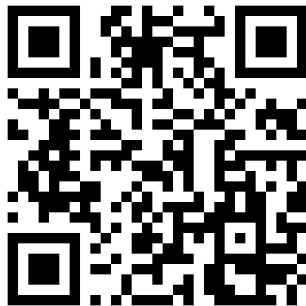
Итог: точность каскада **94,8 %** / макро-F1 0,899, производственная цепочка

Спасибо за внимание

Точность каскада **94,8 %**,
производственной цепочки **91,1 %**,
сокращение стоимости в **14×** раз.

Исходный код, ноутбук §3.3 в исполняемом
виде, демо-комплекс и текст работы — в
репозитории.

Вопросы?



github.com/Qworl/diploma

В1. Отрицательный результат: обучаемый маршрутизатор

Гипотеза Н1: обучаемый стоимостно-чувствительный маршрутизатор превосходит статическое правило хотя бы на одном из трёх бюджетов языковой модели (25 % / 40 % / 50 %).

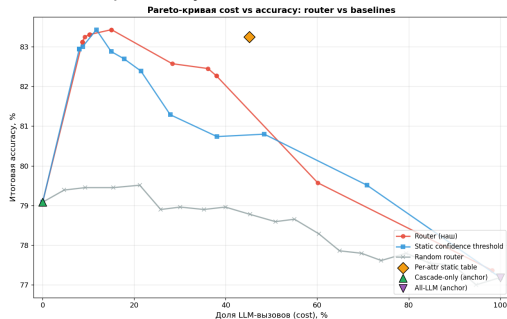
Предрегистрация: git cd9ac7a, 2026-05-13 02:11 (за 1 ч 11 мин до создания артефакта).

Поправка Бонферрони: $\alpha/3 \approx 0,0167$.

Минимально различимый эффект (MDE) при $n = 1539$: $\approx 4,4$ пп (парный McNemar).

Результат: $p > 0,3$ во всех трёх бюджетах.

Парето: обучаемый vs статический



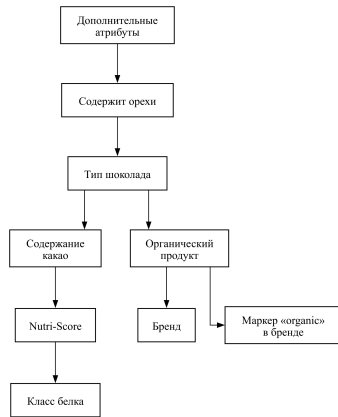
В2. Байесовский слой: причина исключения из production

Что построено. Байесовская сеть на структуре Hill Climb + BIC (на серебряной разметке, 15 тыс. товаров на непересекающихся брендах); условные вероятности доучены на CONCAT(silver, gold-train ×10) с априорной BDeu.

Производственная конфигурация — отключён: байесовский слой принимает 6 % решений с точностью 53 %, что ниже порога замены ML-слоя.

Оставлен как опциональный валидатор.

Активируется поатрибутно, если флаг подозрения превышает калиброванный порог; в текущей конфигурации не задействован.



В3. Серебряный сигнал vs контрольная разметка

Серебряная разметка (консенсус тегов OFF + 3 языковые модели + слепая разметка Orpus) служит **обучающим** сигналом. Контрольная выборка получена по аудит-циклу.

Цикл «аудит — правки экстрактора — переобучение — измерение» воспроизведён для всех 3 категорий с приростом **+18,2, +16,3, +14,4** пп.

Калибровка уверенности — комбинация Platt + изотонической регрессии поатрибутно; ошибка ECE измеряется на отдельной выборке.

В4. Утечки и борьба с ними

Запрет утечки `categories_tags` в эмбединги.

Эмбединги Слой 2 строятся **только** из партнёрских полей: `product_name + brands + ingredients_text + quantity.categories_tags` — это **выход** обогащения, не вход; использование как признака привело бы к утечке целевой переменной.

Разбиение по непересекающимся брендам (60/20/20). Бренды в обучающей, валидационной и контрольной выборках не пересекаются — гарантия обобщения на новые бренды.

Анализ по расстоянию k-NN (§3.3.7.3): точность **не падает** на удалённых от обучающей выборки товарах — монотонный рост от Q1 (82,7 %) до Q4 (91,6 %).

B5. Многоязычность

Языковой состав OFF: $\approx 40\%$ FR, 10% EN, 7% DE, остальное — прочие.

Эмбединг: `paraphrase-multilingual-MiniLM-L12-v2` (50+ языков, общее семантическое пространство).

Поатрибутно-поязыковой срез ($n=3257$ ячеек): взвешенная точность стабильна по основным языкам.

Под балансировку каскад не оптимизировался — это естественный выигрыш многоязычной базовой модели эмбедингов.

В6. Стоимость языковой модели на 1 млн SKU

Каскад делегирует языковой модели лишь 7,1 % ячеек ($n = 3506$) — прямая прокси к стоимости. Сокращение затрат **92,9 % (в 14× раз)** относительно «прямой языковой модели на всё».

Распределение по слоям: L1 (regex высокой точности) — 18,4 %, L2 (ML) — **73,8 %**, L3 (regex низкой точности) — 0,7 %, L4 (языковая модель) — 7,1 %.

Проекция на 1 млн SKU × 22 атрибута ($\sim 2,2 \cdot 10^7$ ячеек):

- ▶ Прямая модель — оплачены все $\sim 2,2 \cdot 10^7$ вызовов.
- ▶ Каскад — оплачены лишь $\sim 1,6 \cdot 10^6$ вызовов (7,1 %).

Топ-3 затратных атрибутов для языковой модели: pasta.grain_type (32,9 %), pasta.pasta_shape (22,8 %), cheeses.is_pdo (13,6 %).

В7. Холодный старт: электроника (§5 ноутбука)

Демонстрация переноса методики на **новую категорию без регулярных правил**.

Электроника (8 атрибутов): смартфоны — RAM, объём памяти, диагональ экрана, поддержка 5G, ...

Перенос: активны только Слой 2 (ML + эмбединги) и Слой 4 (языковая модель); Слой 1 пропускается; Слой 3 — по обучающим парам.

Результат: холодный старт работает на ограниченной обучающей выборке, что подтверждает применимость каскада за пределами продуктового домена.